

Attention Is All You Need 정리

Vaswani et al., 2017

0. Abstract

기존 방식

- RNN / CNN 기반 Encoder-Decoder
- Attention은 보조적으로 사용
- 문제: **sequential computation** → 병렬화 어려움

제안: Transformer

- Recurrence 제거
- Convolution 제거
- Attention만으로 sequence modeling

주요 결과

- WMT14 EN-DE: **28.4 BLEU**
 - WMT14 EN-FR: **41.8 BLEU**
 - 높은 성능 + 빠른 학습 + 높은 병렬화
-

1. Introduction

RNN의 한계

$$h_t = f(h_{t-1}, x_t)$$

- 현재 시점이 이전 hidden state에 의존
- 연산 순차적 수행
- 병렬화 제한
- 긴 sequence 학습 비효율

Transformer의 핵심

기존:

$$RNN/CNN + Attention$$

제안:

$$Attention Only$$

- 모든 token 간 관계 직접 계산
 - long-range dependency 학습 용이
 - 높은 parallelization
-

2. Background

Self-Attention

- 같은 sequence 내부의 token 간 관계 계산
- 거리와 무관하게 dependency 파악 가능

기존 CNN 기반 모델

- 병렬화 가능
- 먼 위치 간 관계 파악에 여러 layer 필요

Transformer

- Self-Attention만으로 모든 위치 연결
 - RNN / CNN 없이 sequence representation 생성
-

3. Model Architecture

Transformer:

$$Encoder \rightarrow Decoder$$

- Encoder: input sequence $\rightarrow$ representation
- Decoder: encoder output을 이용해 output sequence 생성
- Decoder는 **autoregressive**

3.1 Encoder & Decoder

Encoder

6개의 동일 layer

1. Multi-Head Self-Attention
2. Feed-Forward Network

각 sub-layer:

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

- Residual Connection
- Layer Normalization
- $d_{model} = 512$

Decoder

6개의 동일 layer

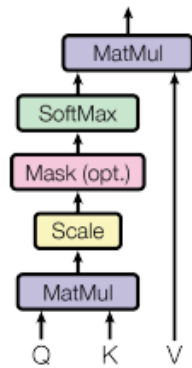
1. Masked Multi-Head Self-Attention
2. Encoder–Decoder Attention
3. Feed-Forward Network

Masking

- 미래 token 참조 금지
 - autoregressive property 유지
-

3.2 Attention

Scaled Dot-Product Attention



Multi-Head Attention

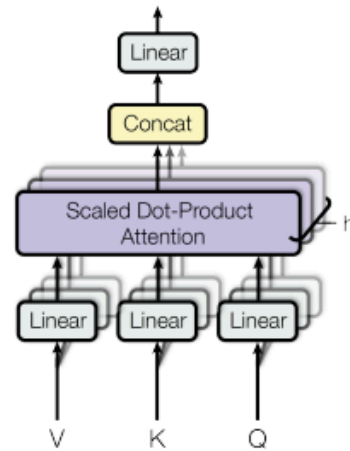


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

- Query Q
- Key K
- Value V

역할

- Query와 Key의 유사도 계산
- Value에 가중치 부여
- 중요한 token 정보 집중

3.2.1 Scaled Dot-Product Attention

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

과정

1. QK^T : 유사도 계산
2. $\sqrt{d_k}$ 로 scaling
3. Softmax $\rightarrow$ attention weight
4. V 와 가중합

Scaling 이유

- d_k 증가 → dot product 크기 증가
- Softmax가 극단적으로 변할 수 있음
- gradient 감소 방지

3.2.2 Multi-Head Attention

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$

논문 설정:

- $h = 8$
- $d_k = d_v = 64$

장점

- 서로 다른 representation subspace 학습
- 여러 종류의 관계를 동시에 파악

3.2.3 Attention의 3가지 형태

Encoder Self-Attention

- $Q = K = V$: Encoder 내부
- 모든 input token 간 관계 계산

Decoder Self-Attention

- Decoder 내부 token 간 관계
- 미래 위치 masking

Encoder-Decoder Attention

- Query: Decoder
- Key / Value: Encoder
- Decoder가 input의 중요한 위치 참조

3.3 Feed-Forward Network

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

구조:

$$512 \rightarrow 2048 \rightarrow 512$$

- 각 position에 독립적으로 적용
- 동일 layer 내에서 parameter 공유

3.4 Embedding & Softmax

- Token $\rightarrow$ Embedding
- Embedding dimension: $d_{model} = 512$

Decoder 마지막:

$$Linear \rightarrow Softmax$$

$\rightarrow$ next-token probability 출력

3.5 Positional Encoding

Attention 자체에는 순서 정보 없음

$$Embedding + Positional Encoding$$

Sinusoidal Encoding

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

역할: token의 위치 정보 주입

4. Why Self-Attention?

① Parallelization

- RNN: $O(n)$ sequential operations

- Self-Attention: $O(1)$

② Long-Range Dependency

- RNN maximum path: $O(n)$
- Self-Attention: $O(1)$
- 먼 token도 한 layer에서 직접 연결

③ Complexity

Self-Attention:

$$O(n^2 d)$$

RNN:

$$O(nd^2)$$

- 일반적인 번역 문제에서 $n < d$ 인 경우가 많아 Self-Attention이 유리
-

5. Training

Optimizer

- Adam
- $\beta_1 = 0.9$
- $\beta_2 = 0.98$

Learning Rate

- Warmup steps: **4000**
- 초기 증가 → 이후 감소

Regularization

- Dropout
 - Label Smoothing
-

6. Results

Translation

EN-DE

28.4 *BLEU*

EN-FR

41.8 *BLEU*

주요 결과

- 기존 모델보다 높은 번역 성능
 - training cost 감소
 - 높은 병렬화
 - 다른 task에도 generalization 가능
-